

MMET383 • LAB #5

Visual Basic: Repetitive Structure

Task 1 • Multiplication Table

Task 2 • Loan Payment Calculator

Multiplication Table — how it is put together

Controls on the form

- TextBox1 / 2 / 3 — base number, low multiplier, high multiplier
- TextBox4 — the table. MultiLine = True, ScrollBars = Vertical, Courier New 9, ReadOnly = True
- Button1 = Calc, Button2 = Clear, Button3 = Exit

What the Calc button does, in order

- 1. Read the three boxes with Integer.TryParse
- 2. Reject zero and negative numbers (the handout asks for this test)
- 3. Reject Low > High, so the range makes sense
- 4. Start the string with a header line
- 5. Do While i <= High: multiply, append one line, i = i + 1
- 6. Assign the finished string to TextBox4

One event handler, one loop, one string. The table is never written line by line to the screen — it is built in memory and assigned once.

The header comment: what each control is

```
' =====  
' Lab #4 MMET383 - Task 1 : Multiplication Table  
' -----  
' Controls on the form (default names, nothing renamed):  
' TextBox1 = Base Number  
' TextBox2 = Low multiplier  
' TextBox3 = High multiplier  
' TextBox4 = output table -> MultiLine = True  
' ScrollBars = Vertical  
' Font = Courier New, 9  
' ReadOnly = True  
' Button1 = Calc Button2 = Clear Button3 = Exit  
' =====
```

Class, variables, and the number check

```
Public Class Form1
```

```
    ' ----- Calc button -----
```

```
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
```

```
        Dim BaseNumber, LowMultiplier, HighMultiplier, Result As Integer
```

```
        Dim i As Integer
```

```
        Dim Message As String
```

```
        ' 1. read the three boxes and make sure they really hold numbers
```

```
        If Not Integer.TryParse(TextBox1.Text, BaseNumber) OrElse
```

```
            Not Integer.TryParse(TextBox2.Text, LowMultiplier) OrElse
```

```
            Not Integer.TryParse(TextBox3.Text, HighMultiplier) Then
```

```
            MessageBox.Show("Please type a whole number in all three boxes.")
```

```
            Exit Sub
```

```
        End If
```

The zero, negative, and range tests

```
' 2. required test: no zero and no negative number
```

```
If BaseNumber <= 0 OrElse LowMultiplier <= 0 OrElse HighMultiplier <= 0 Then
```

```
    MessageBox.Show("All three numbers must be greater than zero.")
```

```
    Exit Sub
```

```
End If
```

```
' 3. the range must make sense
```

```
If LowMultiplier > HighMultiplier Then
```

```
    MessageBox.Show("Low multiplier cannot be larger than high multiplier.")
```

```
    Exit Sub
```

```
End If
```

The Do While ... Loop that builds the table

```
' 4. header line of the table
Message = "Multiplication Table for " & CStr(BaseNumber)

' 5. build the table with a Do While ... Loop
i = LowMultiplier
Do While i <= HighMultiplier
    Result = BaseNumber * i
    Message = Message & vbCrLf &
        CStr(BaseNumber) & " X " & CStr(i) & " = " & vbCrLf & CStr(Result)
    i = i + 1
Loop

' 6. display the finished table
TextBox4.Text = Message
```

Clear, Exit, and the end of the class

```
End Sub

' ----- Clear button -----
Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click
    TextBox1.Text = ""
    TextBox2.Text = ""
    TextBox3.Text = ""
    TextBox4.Text = ""
    TextBox1.Focus()
End Sub

' ----- Exit button -----
Private Sub Button3_Click(sender As Object, e As EventArgs) Handles Button3.Click
    Me.Close() ' "End" also works
End Sub

End Class
```

Loan Payment Calculator — how it is put together

Controls on the form

- TextBox1 / 2 / 3 — loan amount, annual rate (%), term in years
- TextBox4 — the monthly payment, ReadOnly = True
- TextBox5 — the schedule. MultiLine = True, ScrollBars = Vertical, Courier New 8
- Button1 = Payment, Button2 = Amortize, Button3 = Exit
- Form1 AcceptButton = Button1, so the Enter key runs Payment

Three routines, one shared calculation

- GetPayment() — validates the three inputs, then works out MonthlyRate, NumberofPayments and Payment. Returns False if the input cannot be used.
- Button1 Payment — call GetPayment(), show the payment.
- Button2 Amortize — call GetPayment() again, then loop month by month and build the schedule.
- Because both buttons call GetPayment(), Amortize works even if Payment was never clicked.

$\text{Payment} = (\text{LoanAmount} \times \text{MonthlyRate}) / (1 - (1 + \text{MonthlyRate})^{-\text{NumberofPayments}})$. Each month: $\text{Interest} = \text{MonthlyRate} \times \text{Balance}$, $\text{Principal} = \text{Payment} - \text{Interest}$, $\text{Balance} = \text{Balance} - \text{Principal}$.

The header comment: what each control is

```
' =====  
' Lab #4 MMET383 - Task 2 : Loan Payment Calculator  
' -----  
' Controls on the form (default names, nothing renamed):  
' TextBox1 = Loan Amount  
' TextBox2 = Annual Interest Rate (%)  
' TextBox3 = Term (Years)  
' TextBox4 = Payment -> ReadOnly = True  
' TextBox5 = schedule -> MultiLine = True  
' ScrollBars = Vertical  
' Font = Courier New, 8  
' ReadOnly = True  
' Button1 = Payment Button2 = Amortize Button3 = Exit  
' Form1 -> AcceptButton = Button1 (Enter key runs Payment)  
' =====
```

Shared fields and the GetPayment function header

Public Class Form1

' values shared by the Payment button and the Amortize button

Private LoanAmount, AnnualInterestRate, MonthlyRate, Payment As Decimal

Private NumberOfPayments, Term As Integer

' -----

' Read the three input boxes and work out the monthly payment.

' Returns False (after warning the user) if the input is unusable.

' -----

Private Function GetPayment() As Boolean

Validating the three inputs

```
' 1. the boxes must really hold numbers
If Not Decimal.TryParse(TextBox1.Text, LoanAmount) OrElse
    Not Decimal.TryParse(TextBox2.Text, AnnualInterestRate) OrElse
    Not Integer.TryParse(TextBox3.Text, Term) Then
    MessageBox.Show("Please type a number in Loan Amount, Interest Rate and Term.")
    Return False
End If

' 2. zero or negative would divide by zero / make no sense
If LoanAmount <= 0 OrElse AnnualInterestRate <= 0 OrElse Term <= 0 Then
    MessageBox.Show("Loan amount, interest rate and term must all be greater than zero.")
    Return False
End If
```

The payment formula, and the Payment button

' 3. the standard payment formula

$\text{MonthlyRate} = \text{AnnualInterestRate} / (12 * 100)$

$\text{NumberOfPayments} = \text{Term} * 12$

$\text{Payment} = (\text{LoanAmount} * \text{MonthlyRate}) / (1 - (1 + \text{MonthlyRate}) ^ (-\text{NumberOfPayments}))$

Return True

End Function

' ----- Payment button -----

Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click

 If Not GetPayment() Then Exit Sub

 TextBox4.Text = Format(Payment, "0.00")

End Sub

Amortize: locals, recalculation, and the header

```
' ----- Amortize button -----  
Private Sub Button2_Click(sender As Object, e As EventArgs) Handles Button2.Click  
  
    Dim InterestPayment, PrincipalPayment, Balance As Decimal  
    Dim Message As String  
  
    ' recalculate here too, so Amortize works even if Payment was never clicked  
    If Not GetPayment() Then Exit Sub  
    TextBox4.Text = Format(Payment, "0.00")  
  
    ' in the beginning Balance = LoanAmount  
    Balance = LoanAmount  
  
    ' two-line header  
    Message = "Payment" & vbTab & "Total" & vbTab & "Interest" & vbTab &  
        "Principal" & vbTab & "Balance" & vbCrLf &  
        "Number" & vbTab & "Payment" & vbTab & "Payment" & vbTab & "Payment"
```

Amortize: the For ... Next loop

```
' one line per month
For i As Integer = 1 To NumberOfPayments
    InterestPayment = MonthlyRate * Balance ' = (AnnualInterestRate / (12 * 100)) * Balance
    PrincipalPayment = Payment - InterestPayment
    Balance = Balance - PrincipalPayment
    Message = Message & vbCrLf & CStr(i) &
        vbCrLf & Format(Payment, "0.00") &
        vbCrLf & Format(InterestPayment, "0.00") &
        vbCrLf & Format(PrincipalPayment, "0.00") &
        vbCrLf & Format(Balance, "0.00")
Next

' display the repayment schedule
TextBox5.Text = Message
```

Exit, and the end of the class

```
End Sub

' ----- Exit button -----
Private Sub Button3_Click(sender As Object, e As EventArgs) Handles Button3.Click
    Me.Close() ' "End" also works
End Sub

End Class
```